

DecisionRisk Project Source of Truth

v3: End-to-End Product Flow, UI, Runtime, Evaluation, and Operations

This document is the canonical working map for the DecisionRisk final project. It defines the product thesis, domain language, MiroFish integration, artifact contract, LLM verdict council, UI surface, runtime lifecycle, operational behavior, evaluation harness, and implementation roadmap.

Core invariant: MiroFish rehearses plausible futures. DecisionRisk audits the evidence, scores downside, forces adversarial review, and returns a provenance-backed verdict through `verdict.json` and `risk_docket.md`. Raw MiroFish reports are never user-final.

Project: DecisionRisk / MiroFish-first LaunchRisk MVP

Prepared for: final project planning and implementation alignment

Date: May 5, 2026

Status: source-of-truth planning document grounded in the public repo plus project decisions

Table of Contents

1. Executive Product Definition
2. Source-of-Truth Hierarchy
3. Ubiquitous Language
4. Architecture Overview
5. Current Implementation Snapshot
6. DecisionCase Domain Model
7. Risk Pack Layer
8. MiroFish Layer
9. Evidence, Grounding, and ClaimRefs
10. Scenario Ensemble and Metrics
11. LLM Verdict Council
12. Artifact System
13. Product UI Source of Truth
14. End-to-End Run Lifecycle
15. Runtime Mode Contract
16. MiroFish Handoff Contract
17. Job Orchestration, Retries, and Idempotency
18. Artifact Versioning and Migration
19. Council Service Contracts
20. Evaluation and Regression Harness
21. Security, Privacy, and Evidence Handling
22. Observability and Run Telemetry
23. Risk Pack Authoring Contract
24. Human-in-the-Loop Review
25. Post-Decision Monitoring Loop
26. Product Shell, Navigation, and Permissions
27. Deployment and Environment Configuration
28. CI/CD and Release Discipline
29. MVP Definition of Done
30. Open Decisions
31. Source Appendix

1. Executive Product Definition

DecisionRisk is a MiroFish-powered risk decision engine for rehearsing consequential decisions before teams commit to them. The product should not be positioned as a future oracle. It should be positioned as a decision rehearsal, downside discovery, and governance system.

Canonical product sentence: DecisionRisk is a governance layer over multi-agent simulation: MiroFish rehearses plausible futures; DecisionRisk audits evidence, scores downside, forces adversarial council review, preserves dissent, and returns a provenance-backed Risk Docket.

The MVP is the LaunchRisk `ai_memory_launch` demo. It compares three options for a fictional AcmeAI memory launch: `default_on`, `opt_in_beta`, and `enterprise_only`. The default current run is deterministic replay. Live MiroFish execution and live LLM council execution are target capabilities, not completed MVP foundation capabilities.

2. Source-of-Truth Hierarchy

This hierarchy prevents confusion between product design, generated outputs, implementation trackers, and source code.

Rank	Source	Authority
1	Repo source code	Implementation truth for shipped behavior.
2	Generated artifacts	Truth for a completed DecisionRisk run. <code>run_manifest.json</code> is the canonical index.
3	This PDF	Canonical product/architecture/source-of-truth planning doc until superseded by implementation.
4	<code>CONTEXT.md</code>	Canonical domain language and definitions.
5	<code>audit.md</code>	Live implementation tracker and process state, not a generated product artifact.
6	Demo guide	Current walkthrough and known limitations.
7	Chat discussion	Design rationale and iteration history.

Rule: If this PDF and the repo disagree, treat the repo as current implementation truth, then update this PDF or the implementation so the source-of-truth set is consistent again.

3. Ubiquitous Language

Term	Canonical meaning
DecisionCase	Primary DecisionRisk aggregate: question, options, horizon, risk pack, evidence, scenario ensemble, metrics, council verdict, and Risk Docket.
MiroFish Project	Execution substrate used for evidence ingestion, graph construction, simulation, and report substrate behavior.
Risk Pack	Reusable decision-risk template containing ontology, metrics, council roles, scenario patterns, report structure, and validation expectations.
LaunchRisk	MVP Risk Pack for product, pricing, policy, public-announcement, and AI-feature launches.
Risk Docket	Final board-style decision memo containing verdict, evidence, graph findings, scenarios, metrics, council debate, dissent, mitigations, monitoring signals, and audit trail.
ClaimRef	Shared provenance primitive linking assertions to evidence, simulation runs, graph inference, council judgment, confidence, and downstream usage.
run_manifest.json	Root artifact and canonical index for a DecisionRisk run.

4. Architecture Overview

DecisionCase

- > Safety + eligibility gates
- > Risk Pack binding
- > Evidence ingestion + grounding
- > MiroFish Project creation
- > Risk graph/world construction
- > Option x scenario x seed simulation ensemble
- > MiroFish report substrate
- > Deterministic metrics
- > ClaimRef audit
- > Verdict Council
- > verdict.json + council_rounds.json + risk_docket.md
- > manifest validation
- > UI publication

The main product boundary is that DecisionRisk speaks in **DecisionCase** terms. MiroFish remains the simulation substrate. The final user-facing answer must come from the validated DecisionRisk artifacts, not directly from raw MiroFish output.

5. Current Implementation Snapshot

Area	Current status
Repo strategy	AGPL app-first monorepo with MiroFish subtree under <code>apps/decisionrisk-mirofish/</code> . Clean spec package under <code>packages/decisionrisk-spec/</code> .
MVP demo	Deterministic replay for LaunchRisk <code>ai_memory_launch</code> .
Artifacts	Replay outputs include manifest, decision case, evidence, grounding, risk graph, scenario design, scenario runs, metrics, council rounds, verdict, and Risk Docket.
Validation	Validator checks required artifacts, hashes, scenario count, verdict states, non-unsupported ClaimRef support, and Risk Docket sections.
UI	Minimal read-only DecisionRisk artifact APIs and Vue route skeletons exist inside the MiroFish app.
Known gaps	Live MiroFish execution, live LLM council, frontend build verification, full seven-step authoring UI, SQLite index, additional risk packs.

6. DecisionCase Domain Model

A DecisionCase should only be evaluated when it has explicit options and evidence. Open-ended advice without evidence should be blocked.

```
case_id: ai_memory_launch
title: AcmeAI Memory Launch Risk
risk_pack: launch_risk
question: Should AcmeAI launch long-term assistant memory as default-on, opt-in beta, or enterprise-only?
options:
  - default_on
  - opt_in_beta
  - enterprise_only
time_horizon: 30 days
risk_tolerance: low-medium
success_metrics:
  - adoption
  - trust
  - retention
  - press_sentiment
  - regulatory_attention
```

Field	Purpose
case_id	Stable namespace for run outputs.
title	Human-readable case name.
risk_pack	Selects ontology, metrics, scenarios, council roles, thresholds, and docket template.
decision_question	The decision being rehearsed.
options	Mutually comparable actions.
time_horizon	Bounds the simulation window.
risk_tolerance	Governs verdict thresholding.
success_metrics	Declares what matters.
evidence_items	Inputs to graph/world/council grounding.

7. Risk Pack Layer

A Risk Pack packages the reusable decision framework for a category. LaunchRisk is the MVP risk pack.

Risk pack component	LaunchRisk MVP meaning
Ontology	DecisionOption, CustomerSegment, Journalist, Regulator, Competitor, Narrative, Risk, Mitigation.
Scenario templates	Base case, privacy backlash, competitor attack, regulator inquiry.
Metrics	Backlash, adoption friction, narrative cascade, trust damage, regulator attention, competitor exploitability, overall risk.
Council roles	Growth, trust/reputation, regulatory, competitor, chair.
Verdict thresholds	RECOMMEND, DEFER, NO_GO.
Docket template	Board-style decision memo with required sections.
Validation expectations	36 runs, ClaimRefs, manifest hashes, responsible-use notice.

8. MiroFish Layer

MiroFish is the simulation substrate. DecisionRisk should not scatter product behavior across MiroFish internals; it should integrate through a facade layer.

MiroFish capability	DecisionRisk interpretation
Seed upload	Evidence ingestion for a DecisionCase.
Graph/world construction	Risk graph plus stakeholder/world model.
Persona/agent generation	Stakeholder agents such as users, regulators, competitors, press, internal teams.
Simulation	Option x scenario x seed ensemble.
ReportAgent	Raw simulation report substrate, not final answer.
Agent interaction	Future debugging/analysis surface, not MVP-critical.

Integration rule: DecisionRisk product code talks to MiroFish through facades for project, graph, simulation, report, and artifacts. Raw MiroFish outputs are converted into DecisionRisk artifacts and then evaluated by the Verdict Council.

9. Evidence, Grounding, and ClaimRefs

Evidence and grounding are first-class product objects. DecisionRisk should make every material assertion auditable through ClaimRefs.

Grounding level	Meaning	Example
G0	Prompt-only or insufficient evidence	Blocked.
G1	User evidence only	Synthetic/internal fixture docs.
G2	User evidence plus public context summaries	MVP memory launch fixture.
G3	Live public web corroboration	Future web-grounded mode.
G4	Enterprise source-grounded	Internal docs, tickets, CRM, telemetry.

```
{
  "claim_id": "claim_0001",
  "text": "Users may perceive persistent memory as surveillance-like if enabled by default.",
  "claim_type": "risk_signal",
  "status": "source_observed",
  "source_refs": ["evidence:public_privacy_concerns_summary"],
  "confidence": 0.78,
  "used_in": ["simulation_metrics.trust_damage", "verdict.primary_rationale"]
}
```

Unsupported assumptions may be tracked, but they cannot be the sole support for the verdict rationale.

10. Scenario Ensemble and Metrics

The MVP scenario ensemble is deterministic and replayable:

3 options x 4 scenarios x 3 seeds = 36 replay runs

Dimension	Values
Options	default_on, opt_in_beta, enterprise_only
Scenarios	Base case, privacy backlash cascade, competitor attack, regulator inquiry.
Seeds	Three deterministic replay seeds.
Metrics	Backlash intensity, adoption friction, narrative cascade risk, trust damage, regulator attention, competitor exploitability, overall risk score.

Metrics should remain hybrid deterministic in the MVP: structured signals and deterministic formulas generate canonical scores; the LLM council explains, critiques, and synthesizes those scores.

11. LLM Verdict Council

The Verdict Council is product code, not an optional generic LLM council skill. It must run automatically after MiroFish produces a report substrate and the required DecisionRisk artifacts are available.

Council role	Lens
Growth Strategist	Adoption, retention, learning velocity, launch momentum.
Trust & Reputation Analyst	User trust, press framing, privacy perception, brand damage.
Regulatory Analyst	Regulator attention, compliance risk, public commitments.
Competitor Strategist	Adversarial framing, attack surface, market narrative.
Decision Chair	Synthesis, threshold application, final verdict, dissent preservation.

Verdict	Meaning
RECOMMEND	Proceed with the specified option and required mitigations.
DEFER	Do not decide yet; evidence, mitigation, or uncertainty gaps must be closed.
NO_GO	Do not proceed under current conditions.

12. Artifact System

The MVP source of truth is file-first artifacts. SQLite can be added later as a searchable metadata index, but it should not replace the manifest-backed files as the canonical run record.

Artifact	Purpose
run_manifest.json	Canonical artifact index and hashes.
decision_case.json	Normalized decision input.
evidence_manifest.json	Evidence list, types, grounding level.
grounding_report.json	Confidence caps, unsupported assumptions, ClaimRefs.
risk_graph.json	Risk/stakeholder/narrative/mitigation graph.
scenario_design.json	Scenario definitions and metric design.
scenario_runs.json	One record per option x scenario x seed run.
simulation_metrics.json	Deterministic metric outputs.
mirofish_report.*	Raw MiroFish report substrate in live mode.
council_rounds.json	Council deliberation trace.
verdict.json	Machine-readable final verdict.
risk_docket.md	Human-readable board memo.

13. Product UI Source of Truth

The current UI is a minimal read-only artifact viewer inside the MiroFish app. The target UI is a full DecisionRisk workbench that renders artifacts, council reasoning, evidence provenance, and audit status.

13.1 Current routes and files

```
/decisionrisk
/decisionrisk/:caseId

apps/decisionrisk-mirofish/frontend/src/api/decisionrisk.js
apps/decisionrisk-mirofish/frontend/src/views/decisionrisk/DecisionRiskCaseList.vue
apps/decisionrisk-mirofish/frontend/src/views/decisionrisk/DecisionRiskCaseViewer.vue
```

13.2 Current backend API routes

```
GET /api/decisionrisk/cases
GET /api/decisionrisk/cases/:case_id
GET /api/decisionrisk/cases/:case_id/artifacts
GET /api/decisionrisk/cases/:case_id/artifacts/:artifact_name
GET /api/decisionrisk/cases/:case_id/risk-docket
```

13.3 Current viewer contents

UI area	Displays	Source artifact/API
Case list	Available DecisionRisk cases	/api/decisionrisk/cases
Verdict	RECOMMEND, DEFER, or NO_GO	verdict.json
Recommended option	Selected option such as opt_in_beta	verdict.json
Option comparison metrics	Scores across options	simulation_metrics.json
Grounding level	Evidence/source support level	grounding_report.json
Scenario run count	Example: 36 runs	scenario_runs.json
Council dissent	Strongest dissent/minority argument	council_rounds.json and verdict.json
Risk Docket	Rendered board-style Markdown memo	risk_docket.md

13.4 Final target case viewer

Top summary bar

- verdict
- recommended option
- risk pack
- run mode
- validation status
- grounding level

Tabs / sections

1. Executive Summary
2. Option Metrics
3. Scenario Ensemble
4. Evidence & ClaimRefs
5. Council Review
6. Risk Docket
7. Artifact Audit

13.5 Final UI-to-artifact mapping

Screen / tab	Primary content	Source artifacts
Dashboard	Case title, risk pack, final verdict, recommended option, run mode, validation status, grounding, run count, generated time	decision_case.json, verdict.json, run_manifest.json, grounding_report.json, scenario_runs.json
Executive Summary	Verdict badge, recommendation, rationale, mitigations, dissent, responsible-use boundary	verdict.json, risk_docket.md
Option Metrics	Option comparison table and optional chart	simulation_metrics.json
Scenario Ensemble	Filters for option/scenario/seed and run cards	scenario_runs.json
Evidence & ClaimRefs	Evidence list, ClaimRef explorer, source refs, confidence, unsupported assumptions	evidence_manifest.json, grounding_report.json, collected ClaimRefs
Council Review	Council role outputs, peer review, chair synthesis, strongest dissent	council_rounds.json
Risk Docket	Rendered Markdown decision memo	risk_docket.md
Artifact Audit	Manifest, artifact paths, SHA-256 hashes, validation result, raw JSON/Markdown links	run_manifest.json

13.6 Seven-step authoring UI

1. Create DecisionCase
2. Select Risk Pack
3. Add evidence
4. Define options
5. Configure scenarios/seeds
6. Run replay/live
7. Review Risk Docket and council trace

14. End-to-End Run Lifecycle

The lifecycle state machine connects UI states, backend jobs, MiroFish execution, artifact writes, council evaluation, validation, and final publication.

14.1 Canonical live lifecycle

```
draft
-> case_validated
-> evidence_ingested
-> risk_pack_bound
-> scenario_plan_generated
-> mirofish_project_created
-> risk_graph_built
-> scenario_runs_queued
-> scenario_runs_completed
-> simulation_metrics_computed
-> mirofish_report_generated
-> verdict_council_running
-> verdict_generated
-> risk_docket_generated
-> manifest_validated
-> published_to_ui
```

14.2 Canonical replay lifecycle

```
draft
-> case_validated
-> replay_artifacts_generated
-> manifest_validated
-> published_to_ui
```

State	Meaning	UI implication
draft	DecisionCase is being authored but not ready.	Show editable form and missing requirements.
case_validated	Required fields and safety gates passed.	Allow run configuration.
evidence_ingested	Evidence manifest and grounding preparation complete.	Show evidence status and grounding warnings.
scenario_runs_queued	Option x scenario x seed jobs created.	Show queued/running progress.
scenario_runs_completed	All required runs completed or marked failed.	Enable metrics computation.
verdict_council_running	Council services are evaluating artifacts.	Show council progress; do not show final verdict yet.
manifest_validated	Artifact contract passed validation.	Eligible to publish to UI.
published_to_ui	Validated output available in dashboard/viewer.	Show as completed case.

15. Runtime Mode Contract

Mode	Purpose	Uses live MiroFish?	Uses live LLM council?	Source of output
replay	Deterministic demo and CI	No	No	Fixture artifacts
live_smoke	Small live integration test	Yes, minimal	Optional/mock	Live artifacts with reduced scope
live_full	Real product run	Yes	Yes	Live MiroFish + live Council artifacts
eval	Regression testing	Mixed	Mixed	Golden/eval artifacts

Runtime invariants: Replay and live modes must produce the same artifact shape. Live mode must never bypass manifest validation. Live mode must never expose raw MiroFish output directly as the final user answer.

16. MiroFish Handoff Contract

The handoff defines how DecisionRisk controls MiroFish without leaking product semantics into low-level simulation internals.

```
1. create_base_project(decision_case, evidence_manifest)
2. build_graph(project_id, risk_pack, evidence_bundle)
3. clone_project_for_run(project_id, option_id, scenario_id, seed)
4. run_simulation(clone_project_id, scenario_config)
5. collect_simulation_trace(simulation_id)
6. generate_mirofish_report(project_id, completed_runs)
7. normalize_mirofish_report_to_claims(report, traces, graph)
8. write_mirofish_report_artifact(...)
```

Facade	Required responsibility
MiroFishProjectFacade	Create base project, clone per scenario run, track project/run refs.
MiroFishGraphFacade	Build graph/world representation from evidence and risk pack.
MiroFishSimulationFacade	Run option x scenario x seed simulations and return structured traces.
MiroFishReportFacade	Generate raw report substrate and expose report artifact.
MiroFishArtifactFacade	Normalize MiroFish outputs into DecisionRisk artifact format.

17. Job Orchestration, Retries, and Idempotency

Live runs are long-running jobs, not single synchronous requests. The system needs explicit run IDs, status records, event logs, failure artifacts, and retry semantics.

Concern	Canonical decision
Job ID	Use a stable <code>run_id</code> for execution and a separate <code>case_id</code> for the decision.
Idempotency	Same case rerun should create a new run unless explicit overwrite is requested.
Retry	Failed scenario seeds may retry individually without rerunning successful seeds.
Cancellation	A running job should become <code>cancelled</code> and preserve completed artifacts.
Partial output	Partial artifacts may be written but must be clearly marked not validated/final.
Failure artifact	Write <code>run_error.json</code> with stage, error type, message, and retryability.
Resume	Resume from completed artifact checkpoints when hashes and dependencies validate.

17.1 Required operational artifacts

```
run_status.json
run_events.jsonl
run_error.json
```

17.2 Run statuses

```
queued
running
partially_failed
failed
cancelled
completed
validated
published
```

18. Artifact Versioning and Migration

Every artifact and risk pack needs explicit version metadata so future schema changes do not silently invalidate old runs.

```
{
  "manifest_version": "0.1.0",
  "schema_version": "decisionrisk.artifacts.v0",
  "risk_pack_version": "launch_risk.v0.1",
  "generator_version": "decisionrisk-spec@0.1.0"
}
```

Rule	Meaning
Every artifact has <code>schema_version</code>	Validators know which contract applies.
Every risk pack has <code>risk_pack_version</code>	Scores and verdicts can be traced to the exact pack definition.
Every manifest records generator version	Runs can be reproduced or debugged later.
Reject unknown major versions	Prevents silent acceptance of incompatible artifacts.
Provide migration scripts	Allows older replay/live outputs to be upgraded intentionally.

19. Council Service Contracts

The Verdict Council should have explicit service inputs and outputs so it is testable, auditable, and replaceable.

Service	Inputs	Outputs
VerdictCouncilRunner	Manifest plus required artifacts	Council execution package and final outputs.
ReportCritic	MiroFish report, metrics, ClaimRefs	Unsupported claim list, overclaim warnings, missing-scenario warnings.
ClaimRefAuditor	All artifacts	Claim support map and unsupported rationale failures.
VerdictGateEngine	Metrics, safety report, grounding report	Gate result: pass, defer candidate, no-go candidate, or blocked.
CouncilRoleAgent	Role prompt, artifacts, constraints	Role analysis with ClaimRefs, confidence, and dissent.
DecisionChair	Role outputs, gates, metrics	Verdict draft, rationale, mitigations, strongest dissent.
RiskDocketGenerator	Verdict, council, artifacts	<code>risk_docket.md</code> .

19.1 Council config contract

```
council:
  model_policy: deterministic_or_logged
  temperature: 0.2
  max_rounds: 3
  required_roles:
    - growth_strategist
    - trust_reputation_analyst
    - regulatory_analyst
    - competitor_strategist
    - decision_chair
  required_outputs:
    - claim_refs
    - confidence
    - dissent
    - mitigation_requirements
```

20. Evaluation and Regression Harness

DecisionRisk credibility depends on repeatable validation. The evaluation harness should test not only code, but also provenance, safety, council quality, and UI contract behavior.

Eval type	What it checks
Artifact contract eval	Required artifacts exist, validate, and hash-match.
ClaimRef eval	No unsupported primary rationale; material claims link to ClaimRefs.
Safety eval	Disallowed/no-evidence/prompt-only cases are blocked.
Golden replay eval	Replay output remains stable unless intentionally updated.
Council quality eval	Council preserves dissent, cites ClaimRefs, and avoids unsupported certainty.
Metric regression eval	Risk scores do not drift unexpectedly.
UI contract eval	Dashboard/viewer tabs render required artifact fields.
Live smoke eval	One small live MiroFish path works end-to-end.

21. Security, Privacy, and Evidence Handling

Evidence is untrusted input. Prompt-like text inside evidence is never executed. All evidence-derived claims must become ClaimRefs.

Area	Canonical handling
Evidence retention	Define storage duration and deletion behavior before enterprise use.
Sensitive data	Before enterprise use, define whether PII/PHI/secrets are blocked, redacted, or allowed with controls.
Source trust	Label synthetic, internal, public, web-verified, and unsupported sources distinctly.
Prompt injection	Surface warning in run status, Evidence tab, and Risk Docket; treat as quoted evidence only.
Access control	Restrict who can view, author, run, publish, and export artifacts.
Export policy	Separate JSON, Markdown, and PDF export permissions.
Audit trail	Record creator, runner, reviewer, validator, publisher, and timestamps.

Safety boundary: DecisionRisk should not be used as the sole basis for legal, financial, medical, electoral, public-safety, or other high-stakes decisions.

22. Observability and Run Telemetry

Live mode needs operational visibility for debugging, reliability, and cost control.

Telemetry field	Purpose
run_id	Correlates all events/artifacts for one execution.
case_id	Links execution back to the DecisionCase.
mode	replay, live_smoke, live_full, eval.
risk_pack	Shows which pack/version drove the run.
started_at / completed_at / duration	Operational timing.
scenario_count / failed_scenarios	Simulation completeness.
council_round_count	Council execution completeness.
token/cost estimate	Cost governance for live LLM paths.
validation result	Whether output can be published.
artifact hash status	Detects tampering or corruption.

22.1 Logs

```
run_events.jsonl
system logs
council trace logs
MiroFish facade logs
validation logs
```

23. Risk Pack Authoring Contract

Future risk packs must be authored as reusable decision templates, not one-off prompt collections.

```
risk_pack.yaml
ontology.json
scenario_templates.json
metrics.yaml
council_roles.yaml
verdict_thresholds.yaml
docket_template.md
eval_fixtures/
negative_fixtures/
```

Requirement	Why it matters
Ontology	Defines the entities and relationships the risk pack cares about.
Metric definitions	Keeps scores deterministic and explainable.
Scenario templates	Makes stress tests repeatable.
Council roles	Ensures adversarial review is domain-appropriate.
Verdict thresholds	Avoids arbitrary recommendations.
Safety notes	Defines domain boundaries and disallowed uses.
Golden fixture	Enables regression testing.
Negative fixture	Ensures unsafe or unsupported cases are blocked.

24. Human-in-the-Loop Review

DecisionRisk produces a recommendation and Risk Docket. It does not replace the accountable human or organizational decision owner.

```
generated
-> reviewed_by_owner
-> accepted / rejected / revised
-> published
-> monitored_post_decision
```

Field	Purpose
Owner	Person responsible for the decision.
Reviewer	Person/team that reviewed the docket.
Review status	Draft, reviewed, accepted, rejected, revised.
Review notes	Human judgment outside the model output.
Final human decision	Whether the organization followed the recommendation.
Outcome follow-up	What actually happened later.

25. Post-Decision Monitoring Loop

The Risk Docket should include mitigations and monitoring, but the product should also support attaching observed outcomes after the decision.

```

Risk Docket generated
-> decision made
-> launch/action occurs
-> monitoring signals collected
-> observed outcome attached
-> case retrospective generated
-> risk pack improved

```

Artifact	Purpose
decision_record.json	Captures the final human decision and reviewer metadata.
monitoring_plan.json	Turns docket monitoring signals into trackable checks.
observed_outcomes.json	Records what actually happened.
retrospective.md	Compares predicted risks to observed outcomes and updates lessons.

26. Product Shell, Navigation, and Permissions

The UI section defines what screens show. The product shell defines how people reach those screens and who can do what.

Question	Canonical product decision
Where does DecisionRisk appear?	Add a DecisionRisk entry in the MiroFish navigation and optionally a project detail tab.
Who can create a DecisionCase?	Analyst/owner role by default; admin can configure.
Who can run live mode?	Restricted because live mode may consume cost and external services.
Who can publish a Risk Docket?	Owner or reviewer role only.
Can artifacts be downloaded?	Yes, subject to export permissions for JSON, Markdown, and PDF.
Can users compare runs?	Yes, future UI should compare replay vs live or before/after mitigation runs.
Can users rerun with changed mitigations?	Yes; this is core product value.

27. Deployment and Environment Configuration

Environment	Behavior
Local replay	No API keys; Python standard library package; outputs directory as source of truth.
Local MiroFish app	Flask backend, Vue frontend, DECISIONRISK_OUTPUTS_DIR .
Live mode	MiroFish dependencies, LLM provider keys, storage configuration, job runner, optional SQLite index.

27.1 Environment variables

```
DECISIONRISK_OUTPUTS_DIR  
DECISIONRISK_MODE  
DECISIONRISK_ALLOWED_RISK_PACKS  
DECISIONRISK_LLM_PROVIDER  
DECISIONRISK_LLM_MODEL  
DECISIONRISK_ENABLE_LIVE_MIROFISH  
DECISIONRISK_ENABLE_WEB_GROUNDING
```

28. CI/CD and Release Discipline

28.1 Required CI before merge

- Python replay generation.
- `decisionrisk validate`.
- Unit tests.
- `compileall`.
- Safety negative fixtures.
- Frontend build once dependencies are installed.
- Artifact schema validation.
- Golden fixture diff.

28.2 Required release artifacts

- Source-of-truth PDF.
- Demo artifacts.
- Validation report.
- Changelog.
- Risk pack version notes.

29. MVP Definition of Done

1. Replay path generates valid artifacts.
2. Live smoke path creates at least one MiroFish-backed run.
3. Verdict Council runs automatically after MiroFish output.
4. Final answer is always from `verdict.json` or `risk_docket.md`, never raw MiroFish output.
5. UI shows dashboard, case viewer, metrics, council, ClaimRefs, docket, and artifact audit.
6. Validator passes in CI.
7. Safety negative fixtures pass.
8. Frontend build passes.
9. Demo script works end-to-end.
10. This source-of-truth doc matches implementation.

30. Open Decisions

Open decision	Default recommendation
Should live runs write partial artifacts?	Yes, but mark them non-final and non-validated.
Should live council be deterministic?	Use low temperature and log model/provider/config; deterministic replay remains the audit baseline.
Should SQLite become source of truth?	No. Use it as metadata/search index only.
Should UI authoring ship in MVP?	Not before read-only viewer, artifact audit, and live smoke path are stable.
Should additional risk packs ship before LaunchRisk is complete?	No. Stabilize LaunchRisk contract first, then clone the authoring pattern.

31. Source Appendix

This document is grounded in the public DecisionRisk repository and current project decisions. Key source anchors:

- **README.md**: DecisionRisk product definition, AGPL/MiroFish-first monorepo, MVP replay status, artifact contract, ClaimRef provenance, licensing boundary.
- **CONTEXT.md**: canonical domain language for DecisionCase, MiroFish Project, Risk Pack, LaunchRisk, Risk Docket, run_manifest.json, ClaimRef, and audit.md.
- **demo/README.md**: deterministic replay guide, generated artifacts, validation checks, ClaimRefs, 36-run ensemble, current APIs, Vue routes, current viewer contents, known UI/live limitations.
- **audit.md**: foundation MVP completed work, remaining live MiroFish facade work, mandatory Verdict Council invariant, required services/gates, frontend build limitation, future authoring UI, SQLite index, additional risk packs.
- **tasks/todo.md**: completed foundation, replay MVP, tests, and MiroFish integration tasks.
- **fixtures.py / CLI**: deterministic LaunchRisk fixture payloads, validation behavior, ClaimRef and artifact enforcement behavior.